

Lösungsheft

Tag 6

Musterlösungen – BPMS, APIs,
Cloud-native und Plattform-Strategie.

Musterlösungen · nicht die einzige Antwort

Hinweis:

Musterlösungen sind Diskussionsbasis,
nicht die einzige korrekte Lösung.

Begleitend:

Aufgabenheft & Lehrskript Tag 6

Dozent:

Can Siebert

Niveau-Mix:

3 L1 · 5 L2 · 3 L3

Hinweise zu den Musterlösungen

Die folgenden Musterlösungen sind *eine* mögliche, didaktisch nachvollziehbare Lösung. Heute besonders wichtig: *Plattform-Entscheidungen sind 6+ Monate ändert* – die Lösung muss daher zwingend *Annahmen, verworfene Alternativen* und *Frühwarnsignale* dokumentieren.

Nutzung im Coaching

Prüfen Sie Ihre Lösung gegen drei Fragen: (1) Welche Entscheidung ist in 6 Monaten teuer zu ändern? (2) Welche habe ich bewusst getroffen, welche per Default? (3) Welches Frühwarnsignal würde mich zur Kurskorrektur zwingen?

Niveau L1 – Wissen & Reproduktion

Hilfsvideos zu diesem Tag

Die folgenden YouTube-Erklärvideos vermitteln die Inhalte, die Sie zur Bearbeitung der Aufgaben benötigen. Klicken Sie auf den Titel, um das Video direkt zu öffnen; die vollständige URL steht in Klammern.

1. Was ist BPM? — Business Process Management in 2 Minuten

<https://youtu.be/9NXq0a-X7cI>

(URL: <https://youtu.be/9NXq0a-X7cI>)

2. Was ist eine REST-API? — Einführung in REST APIs

https://youtu.be/ZuINS_-n-AM

(URL: https://youtu.be/ZuINS_-n-AM)

3. Monolithen vs. Microservices — wann ist welche Architektur sinnvoll?

<https://youtu.be/N0k3VKL4Als>

(URL: <https://youtu.be/N0k3VKL4Als>)

4. Business Process Management — Closed-Loop einfach erklärt

<https://youtu.be/IBenKthHmUU>

(URL: <https://youtu.be/IBenKthHmUU>)

Tipp: Öffnen Sie das Video, lassen Sie es im Hintergrund laufen und arbeiten Sie parallel an der Aufgabe.

Aufgabe 1 – BPMS-Bausteine

L1 • Wissen

~ 8 min

YouTube-Suchquery: BPMS Bausteine Prozessengine Workflow Engine Closed Loop Camunda

Musterlösung

Sieben Kernbausteine:

1. *Prozessengine*: führt das Modell aus. Risiko ohne: alles bleibt Skizze.
2. *Aufgaben-/Worklist*: stellt Tasks an User. Risiko: keine Sichtbarkeit, was zu tun ist.
3. *Regelwerk (DMN)*: Entscheidungslogik außerhalb des Prozesses. Risiko: Logik im Code versteckt.
4. *Forms*: Datenerfassung im Prozess. Risiko: Excel-Workarounds.
5. *Monitoring / Dashboards*: Real-time Steuerung. Risiko: blinde Steuerung.
6. *Audit Trail*: revisionssichere Historie. Risiko: keine Audit-Fähigkeit.
7. *Versionierung*: parallele Modellstände. Risiko: Roll-back unmöglich.

Closed Loop: *Modellieren* (BPMN-Soll) □ *Ausführen* (Engine + Tasks) □ *Überwachen* (KPIs + Mining) □ *Optimieren* (Change Request, neue Version). Ohne Überwachen+Optimieren ist es nur “Run” – kein Loop.

Aufgabe 2 – API als Vertrag

L1 • Wissen

~ 10 min

YouTube-Suchquery: API Contract REST Endpoint Versioning Auth Rate Limiting Best Practice

Musterlösung

Acht Pflichtbestandteile:

1. *Endpoint* – URL + HTTP-Methode. Fehlt: Aufruf nicht reproduzierbar.
2. *Input* – getypte Felder + Pflicht/Optional. Fehlt: “mal mit, mal ohne” □ Engineering hölle.
3. *Output* – Struktur + Typ + Beispiele. Fehlt: Konsument interpretiert frei.
4. *Fehlercodes* – klare Liste (4xx vs. 5xx). Fehlt: Retry / manuell nicht entscheidbar.
5. *Auth* – OAuth, mTLS, API-Key. Fehlt: Security-Gap, Audit-Versagen.
6. *Rate Limits* – Anfragen pro Zeit. Fehlt: Lieferantenseite kann Sie blockieren.
7. *Versionierung* – /v1/ oder Header. Fehlt: “Breaking Change am Mittwoch” □ Ausfall.
8. *Monitoring / Correlation-ID* – Trace-Header. Fehlt: Fehlersuche unmöglich.

Eigentümer: der *Anbieter*. Er verantwortet Stabilität, Versionierung und Deprecation. Der Konsument darf den Vertrag nur konsumieren – nicht neu definieren.

Aufgabe 3 – Sync vs. Async, Idempotenz, Retry

L1 • Wissen

~ 8 min

YouTube-Suchquery: Sync vs Async API Event Driven Architecture Idempotency Retry Exponential

Musterlösung

1. Sync vs. Async: *Sync* = Anfrage wartet auf Antwort; Aufrufer weiß sofort, was passiert. *Async* = Anfrage geht raus; Antwort kommt später (Event/Webhook). *Sync* ist gut bei interaktiven UI-Pfaden; *Async* ist gut bei langlaufenden, fehleranfälligen Prozessen.

2. Idempotenz: Mehrfacher Aufruf hat dieselbe Wirkung wie ein einzelner. Beispiel: PUT mit ID = idempotent; POST ohne ID = nicht idempotent. Pflicht für Retry: ohne Idempotenz erzeugt jeder Retry potenziell eine *neue* Wirkung (Doppelbuchung).

3. Retry mit exponential backoff: Wartezeiten verdoppeln sich pro Retry (z. B. 1 s, 2 s, 4 s, 8 s). Schutz vor “Cascading Failure” bei überlastetem Downstream. *Nicht retryen:* bei Side-Effects ohne Idempotenz, bei 4xx-Errors (Client-Fehler), bei Auth-Fail.

4. Correlation-ID: Bindet alle Async-Schritte an einen Case. Ermöglicht Fehleranalyse und Audit. Pflicht-Header für jeden Aufruf.

Vergleichstabelle:	Aspekt	Sync	Async
	Antwort	sofort	später (Event)
	Kopplung	eng	lose
	Resilienz	gering (Kaskade)	hoch (Pufferung)
	Komplexität	gering	höher (Correlation)

Niveau L2 – Anwendung & Transfer

Aufgabe 4 – BPMS-Blueprint *Customer Complaint Handling*

L2 • Anwendung

~ 25 min

YouTube-Suchquery: BPMS Blueprint Customer Complaint Workflow SLA Eskalation Camunda

Musterlösung**BPMS-Blueprint:****Start:** *Reklamation eingegangen* (Message via E-Mail-Gateway oder CRM).

- User Task (Service Agent): *Klassifizieren* (Kulanz / Defekt / Lieferung). SLA 4 h. System: CRM.
- **XOR:** *Rückfrage nötig?*
 - Ja ☐ User Task (Service Agent): *Rückfrage stellen*. Timer 48 h. Eskalation nach Ablauf.
 - Nein ☐ weiter.
- User Task (Service Lead): *Entscheidung Kulanz/Retoure*. Regel DMN (Schwellenwert + Kundenstatus). SLA 2 Tage.
- Service Task: *Abwicklung im ERP* (Gutschrift / Rücknahme).
- Service Task: *Kundenkommunikation* (E-Mail). Audit-Log.
- End: *Reklamation abgeschlossen* + Reporting (KPI-Update).

SLAs / Eskalation: 48 h Erstreaktion, 10 T Abschluss. Bei Überschreitung Eskalation an Service Lead.**Drei KPIs:**

- *SLA-Einhaltung* (% Fälle in SLA) – Owner: Service Lead.
- *Rework-Quote* (% Fälle mit Nachbearbeitung) – Owner: QM.
- *Customer Satisfaction nach Reklamation* – Owner: VoC-Team.

Entscheidung unter Unsicherheit: *Im BPMS muss:* Klassifizierung + SLA + Eskalation. *Außerhalb:* VoC-Befragung (eigenes Tool, kein Workflow-Step) – sonst werden Kunden zu früh kontaktiert, die KPI verzerrt.

Aufgabe 5 – API-Verträge

L2 • Anwendung

~ 22 min

YouTube-Suchquery: REST API Contract Order Status Return GetOrderStatus CreateReturn Error

Musterlösung

API 1: GetOrderStatus (sync)

- Endpoint: GET /v1/orders/{order_id}/status.
- Input: order_id (Pfad), X-Correlation-ID (Header).
- Output: status, last_event_at, carrier, tracking_id.
- Fehler: 404 (Order unknown), 401/403 (Auth), 503 (Backend down).
- Auth: OAuth2 client_credentials.
- Versionierung: /v1/.
- Monitoring: Correlation-ID + Response-Time-Logging.

API 2: CreateReturn (async via Event)

- Endpoint: POST /v1/returns (sync acknowledgement) + Event ReturnCreated (async).
- Input: order_id, reason_code, items[], customer_contact.
- Output (sync): return_id, acknowledged_at; (async event): status_change.
- Fehler: 400 (Pflichtfeld fehlt), 409 (Order nicht reklamierbar), 503.
- Auth: OAuth2 + Idempotenz-Key.
- Versionierung: /v1/.
- Monitoring: Correlation-ID + Event-Auditing.

Drei Fehlerstrategien:

- *Timeout 503*: 3× Retry mit exponential backoff; danach Eskalation an Service Lead.
- *Daten fehlen 400*: Sofort manuell (User Task “Pflichtfeld ergänzen”).
- *Auth-Fail 401/403*: keine Retry; sofortige Eskalation an IT-Security.

Sync/Async:

- *GetOrderStatus* = sync (UI braucht sofortige Antwort).
- *CreateReturn* = async (langlaufender Prozess, mehrere Status-Änderungen). Sync-Acknowledgement zur sofortigen Bestätigung, dann Event-Stream.

Aufgabe 6 – Cloud-native Betriebsanforderungen

L2 • Anwendung

~ 20 min

YouTube-Suchquery: Cloud Native Twelve Factor Auto Scaling Observability Circuit Breaker SLO

Musterlösung**Fünf Anforderungen:**

1. *Auto-Scaling (horizontal)*: Container-Anzahl skaliert mit Last; sichtbar im Peak-Saison-Verhalten. Begründung: Skalierungs-Bedarf ist $10\times$ in der Peak.
2. *Observability*: *Logs* (strukturiert, JSON), *Metrics* (Latenz, Error-Rate, Throughput), *Traces* (Distributed Tracing pro Case-ID).
3. *Resilienz*: *Circuit Breaker* verhindert Kaskaden bei Downstream-Ausfall; *Retry mit Backoff* fängt transiente Fehler ab.
4. *Deployment*: *CI/CD* mit Staging; *Blue/Green* für Zero-Downtime; *Rollback* in < 5 Minuten.
5. *Security (Zero Trust)*: *mTLS* zwischen Services; *Secrets Management* (kein Klartext im Repo); Least-Privilege-Berechtigungen.

Anti-Muster “Skalierung ohne Observability”:

Wenn Container im Peak auf 20 hochskaliert, aber niemand Latenz, Error-Rate und Traces sieht, ist ein Ausfall im 10. Container unsichtbar – bis Kunden anrufen. Observability ist die einzige Sicherheit, dass Skalierung tatsächlich *Wirkung* erzielt und nicht nur *Kosten* produziert.

Aufgabe 7 – System-of-Record-Entscheidung

L2 • Anwendung

~ 18 min

YouTube-Suchquery: System of Record Data Ownership Master Data Management CRM ERP Conflict

Musterlösung**1. System of Record pro Datenobjekt:**

- **Kunde:** CRM (vertriebsseitig, vollständigste Stammdaten).
- **Auftrag:** ERP (rechnungs-/buchhalterisch verbindlich).
- **Reklamation:** BPMS (Workflow-Case ist der “Master-Case”).

2. Konflikt bei Datenobjekt Kunde:

Wenn CRM eine neue Lieferadresse führt, das ERP eine “alte” Adresse aus einer früheren Rechnung, gilt: *CRM ist Master* → ERP wird über Event *CustomerAddressUpdated* aktualisiert. Bei manuellen Änderungen im ERP wird ein *Konflikt-Event* erzeugt, Data Steward muss entscheiden.

3. Master-Slave-Regel:

Master-System publiziert Events; alle Slave-Systeme konsumieren. Bei manuellen Änderungen am Slave: *Event back to Master* (“Pull-Request”); Master Steward entscheidet.

4. Bewusst akzeptiert vs. nie:

- **Akzeptiert (mit Guardrail):** bis zu 4 h Verzögerung bei Kundendaten-Sync ist OK – solange ein End-User-sichtbares Banner im ERP zeigt “Daten werden synchronisiert”. Guardrail: nach 4 h muss Alerting kommen.
- **Nie:** Inkonsistenz beim *Reklamationsstatus* – sonst bekommen Kunden falsche Auskunft. Hier ist Real-time-Sync verpflichtend.

Aufgabe 8 – E2E-Methodenintegration: Model → Run → Improve

L2 • Anwendung

~ 18 min

YouTube-Suchquery: Process Model Run Improve Closed Loop BPMS Mining KPI Continuous Improvement

Musterlösung

Model: *BPMN* als Soll (ausführbar im BPMS) + *VSM* als End-to-End-Übersicht für Engpässe. *BPMN* steuert Ausführung, *VSM* steuert Optimierung.

Run: *BPMS-Engine* führt den *BPMN*-Prozess aus; *REST-API* zum *CRM/ERP* integriert; *Event-Bus* für asynchrone Schritte.

Improve: *Process Mining* auf den Audit-Logs des BPMS □ *Discovery* zeigt Ist-Pfade + Varianten; *KPI-Dashboard* (SLA-Einhaltung, Rework, CSAT) im Review-Zyklus.

Process Review Board:

- Rollen: Process Owner, IT-Lead, Operations-Lead, Data Steward, Compliance-Lead.
- Frequenz: monatlich, 90 Min.
- Entscheidet: Change Requests zum *BPMN*, KPI-Anpassungen, Audit-Themen.

Faustregel: Der Loop ist *geschlossen*, sobald eine im Mining entdeckte Abweichung in einem dokumentierten Change Request endet – und dieser zu einer neuen *BPMN*-Version führt, die wieder gemessen wird. Wenn drei Reviews kein Change Request produzieren, ist es *kein* Loop, sondern Reporting.

Niveau L3 – Management & Entscheidungsarchitektur

Aufgabe 9 – Fallstudie *EuroLogix*

L3 • Management

~ 45 min

YouTube-Suchquery: Logistics Exception Handling BPMS Cloud Native API Integration Roadmap

Musterlösung**1. BPMS-Prozess:**

Start “Exception erkannt” (Event aus TMS) → Task *Klassifizieren* → XOR “kritisch?” (Ja → sofortige Eskalation + Kundeninfo; Nein → normal) → Task *Ursache klären* (CRM/TMS) → Task *Maßnahme* (Umroute / Neuversand) → Task *Kundenkommunikation* → End “geschlossen” + Reporting. SLAs: Erstreaktion 2 h, Lösung 24 h, Eskalation nach 4 h an Duty Manager.

2. Integrationspunkte:

1. *Event ExceptionRaised* (async) mit Case-ID, Shipment-ID, Timestamp, Severity.
2. *API GetShipmentStatus* (sync); Fallback Cache/Manuell bei Timeout.
3. *API NotifyCustomer / Event CustomerNotified*; Audit-Logging Pflicht.

3. Cloud-native (4 Punkte): Auto-Scaling bei Peak; zentrale Logs/Metrics/Traces; Circuit Breaker + Retry; Blue/Green-Deployment.

4. Methodenintegration: BPMN als Soll, VSM für Engpässe, KPIs als Steuerung, Mining für Varianten → monatliches Process Review Board (Owner + IT + Ops).

5. MVP (Release 1): *Drin:* BPMS-Case-Handling + 1 Event + 1 Status-API + SLA/Eskalation + Dashboard.

Bewusst nicht: Full Data Warehouse Rebuild; Predictive Modelle; mobile App für Driver. Begründung: 9 Wochen sind zu kurz, Risiko zu hoch; alle drei können in Release 2 nachgezogen werden, ohne MVP umzubauen.

Aufgabe 10 – Architekturpfad: event-driven vs. sync

L3 • Management

~ 25 min

YouTube-Suchquery: Event Driven Architecture vs Synchronous Trade Off Resilience Audit

Musterlösung

Trade-off-Matrix:

Dimension	Event-driven	Überwiegend sync
Resilienz	hoch (Pufferung)	gering (Kaskade)
Audit / Nachverfolg.	schwieriger (Event-Order)	einfacher (lineare Aufrufkette)
Skalierung	sehr gut (lose Kopplung)	limitiert durch synchrone Wartezeiten
Komplexität	hoch (Correlation, Order, Idempotenz)	niedrig
Team-Skills	verlangt Senior-Engineering	breiter verfügbar

Entscheidung für *EuroLogix*:

Überwiegend event-driven mit sync-Inseln. Begründung: Peak-Saison verlangt Pufferung; Shipment-Lifecycle ist von Natur aus eventbasiert. Reine Sync-Architektur würde bei Peak kollabieren. Sync bleibt nur für UI-getriebene Lookups.

Sync-Inseln:

- *GetShipmentStatus* (UI braucht sofort Antwort).
- *Customer-Notification-Status-Check* (Service-Agent fragt im Gespräch).

Frühwarnsignal zur Umkehr:

Wenn nach 3 Monaten die durchschnittliche Event-Latenz über 2 Sekunden steigt oder das Team für jeden neuen Use Case 4+ Tage Setup-Zeit braucht – dann ist event-driven für dieses Team zu komplex. Rückweg: BPMS-Engine als Orchestrator weiterhin behalten, Events nur für nötige Pufferung.

Aufgabe 11 – Transferprojekt – E2E Integration & BPMS Platform Strategy

L3 • Management

~ 45 min

YouTube-Suchquery: Platform Strategy BPMS Integration Cloud Native API Governance CDO

Musterlösung

1. Plattform-Entscheidungsarchitektur:

- *Zentral*: API-Standards (OpenAPI, Auth-Pattern), Observability-Stack, Security-Standards, BPMN-Style-Guide.
- *Dezentral*: Team-Backlog, Service-Implementation, Tech-Stack innerhalb der Standards.

2. API Governance:

- Versionierung: $/v\{N\}/$ im Pfad; Deprecation 12 Monate angekündigt.
- SLA/SLO pro API (Latenz P95, Verfügbarkeit 99,5 %).
- Ownership: jede API hat *einen* Team-Owner.
- Contract Testing: Konsumenten testen gegen Pact-ähnliche Specs in CI.
- Change-Prozess: API-Review-Board, mind. 2 Wochen vor Release.

3. BPMS Governance:

- Prozess-Lifecycle: Draft → Approved → Retired (siehe Tag 4).
- Release-Management: monatlich, Hotfix on demand.
- Exception-Policy: max. 10 % Manual-Exit pro Prozess.
- Audit Trail: 7 Jahre, suchbar.
- KPI-Steuerung: SLA-Einhaltung, Rework, CSAT pro Prozess.

4. Cloud-native Betriebsmodell:

- SLOs: 99,5 % Verfügbarkeit Tier-1 Services; P95 Latenz < 500 ms.
- Skalierungsstrategie: Auto-Scaling auf CPU + Custom Metrics (Queue-Länge).
- Incident Management: PagerDuty-ähnlich, On-Call-Rotation, Post-Mortem-Pflicht.
- FinOps light: monatliches Kosten-Review pro Service-Team; Budget-Alerts.

5. Roadmap (16 Wochen, 3 Releases):

- *Rel 1 (Wo 1–6, MVP)*: 1 Prozess, 2 APIs, 1 Event, 1 Dashboard, Audit Trail.
- *Rel 2 (Wo 7–12, Skalierung)*: 3 Prozesse, 5 APIs, Auto-Scaling, Observability-Stack.
- *Rel 3 (Wo 13–16, Sustain)*: Governance Board, Incident-Modell, FinOps.

Messkonzept: Leading = Lead Time pro Change; Lagging = Anzahl Incidents, Anteil SLA-Verletzungen, Audit-Findings.

6. Entscheidungen unter Unsicherheit:

1. *Architekturpfad*: überwiegend event-driven mit sync-Inseln (siehe Aufgabe 10). Frühwarnsignal: Event-Latenz > 2 s, Setup-Zeit > 4 T pro Use Case.
2. *MVP-Scope = 2 Integrationen*: GetShipmentStatus + ExceptionRaised-Event. *Stop-Doing*: keine Customer-Mobile-App, kein Real-time-ETL ins DWH, keine ML-Komponente in Release 1. Begründung: jede dieser drei Optionen verschwendet das 9-Wochen-Fenster.

Abschluss

Die Musterlösungen sind eine Diskussionsbasis. Heute prüfen Sie besonders: Habe ich BPMS, API und Cloud-native als *drei Teile einer Plattform-Entscheidung* verstanden – oder als isolierte Themen?

Nächster Schritt

Bringen Sie ins Coaching: Welche API gilt bei Ihnen heute als “Vertrag mit Versionierung”? Welche führt heute ein Schattendasein – und welcher Audit würde sie sichtbar machen?